Homework 4

Python `selenium`, APIs, & Pandas Basics

AUTHOR
Byeong-Hak Choe

PUBLISHED
November 24, 2025

MODIFIED
November 24, 2025

Direction

- Please submit your **Python Script** for **Part 1 in Homework 4** to Brightspace with the name below:
    - `danl_299_hw4_LASTNAME_FIRSTNAME.py`
      (e.g., `danl_299_hw4_choe_byeonghak.py`)
- The due for Part 1 in Homework 4 is December 8, 2025, 5:15 P.M.
- The due for Part 2 in Homework 4 is February 2, 2025, 5:15 P.M.
- Please send Prof. Choe an email (bchoe@geneseo.edu) if you have any questions.

Part 1. Data Collection

Question 1

↑ Back to top

- Go to the following webpage: https://[illegible]e/finance/property/property-annualized-sales-update.page

- Provide a Python Selenium code to download all the MS Excel files in the webpage above.
    - Please make sure that you do not download any PDF files.

Question 2

The `series_housing_price` DataFrame contains FRED series `id`s for the Home Price Index across 16 metropolitan statistical areas, each categorized into three price tiers: low, middle, and high.

```
url = 'https://bcdanl.github.io/data/fred_api_series_housing_price.csv'
series_housing_price = pd.read_csv(url)
```

Show 10 entries Search:

	id	title	observation_start	observation_end	frequency	f
1	ATXRHTSA	Home Price Index (High Tier) for Atlanta, Georgia	1991-01-01	2025-01-01	Monthly	
2	ATXRLTSA	Home Price Index (Low Tier) for Atlanta, Georgia	1991-01-01	2025-01-01	Monthly	
3	ATXRMTSA	Home Price Index (Middle Tier) for Atlanta, Georgia	1991-01-01	2025-01-01	Monthly	

↑ Back to top

	id	title	observation_start	observation_end	frequency	f
4	BOXRHTSA	Home Price Index (High Tier) for Boston, Massachusetts	1987-01-01	2025-01-01	Monthly	I
5	BOXRLTSA	Home Price Index (Low Tier) for Boston, Massachusetts	1987-01-01	2025-01-01	Monthly	I
6	BOXRMTSA	Home Price Index (Middle Tier) for Boston, Massachusetts	1987-01-01	2025-01-01	Monthly	I
7	CHXRHTSA	Home Price Index (High Tier) for Chicago, Illinois	1992-01-01	2025-01-01	Monthly	I
8	CHXRLTSA	Home Price Index (Low Tier) for Chicago, Illinois	1992-01-01	2025-01-01	Monthly	I
9	CHXRMTSA	Home Price Index (Middle Tier) for Chicago, Illinois	1992-01-01	2025-01-01	Monthly	I
10	DNXRHTSA	Home Price Index (High Tier) for	19	2025-01-01	Monthly	I

↑ Back to top

id	title	observation_start	observation_end	frequency	f
	Denver, Colorado				

Showing 1 to 10 of 48 entries Previous 1 2 3 4 5 Next

For more information regarding the index, please visit Standard & Poor's. There is more information about home price sales pairs in the Methodology section. Copyright, 2025, Standard & Poor's Financial Services LLC.

Provide a Python code (**excluding your FRED API key**) to construct the following DataFrame:

Show 10 entries Search:

	city	state	tier	date	home_price_index
1	Phoenix	Arizona	Low	2025-01-01	395.596961829852
2	Phoenix	Arizona	Middle	2025-01-01	334.960782301608
3	Phoenix	Arizona	High	2025-01-01	328.103589121062
4	Phoenix	Arizona	Low	2024-12-01	393.6955239686
5	Phoenix	Arizona	Middle	2024-12-01	333.230609005061
6	Phoenix	Arizona	High	2024-12-01	324.796991180919
7	Phoenix	Arizona	Low	2024-11-01	390.470049983723
8	Phoenix	Arizona	Middle	2024-11-01	331.585633481799
9	Phoenix	Arizona	High	2024-11-01	321.623169348865
10	Phoenix	Arizona	Low	2024-10-01	387.160576441941

Showing 1 to 10 of 21,072 entries

Previous 1 ↑ Back to top 4 5 … 2,108 Next

- Note that the `id` variable in the `series_housing_price` DataFrame corresponds to the `series_id` key used in the parameter dictionary (`param_dicts`) used for making requests to the FRED API.

- To iterate over the `id` values in the `series_housing_price` DataFrame, you can convert the variable to a list using `.tolist()`:

```
for val in series_housing_price['id'].tolist():
   # do something with val
```

- Below replaces 'Home Price Index (' with '' (empty string) in the `title` column:

```
df_all['title'] = df_all['title'].str.replace('Home Price Index (', '')
```

- The `title` column can split into the `tier` and `city` variables using the following code:

```
# Split the 'title' column into two new variables, 'ranking' and 'title'
# by the first occurrence of " Tier) for ".

# expand=True tells pandas to return the results in a DataFrame
  # If expand were False, the result would be a Series of lists.

# Use \) to escape the closing parenthesis, which has special meaning in regex
df_all[['tier', 'city']] = df_all['title'].str.split(' Tier\) for ', expand=True)
```

- The DataFrame is sorted first by `state`, then by `city` within each `state`, then by `date` in descending order within each `city`, and finally by `tier` within each `city-date` group.
  - To sort observations in a custom order (like Low, Middle, High), you can convert the `tier` variable to a categorical variable with an explicit order, and then sort.

```
df = pd.DataFrame({
    'tier': ['Middle', 'High', 'Low', 'Middle', 'Low'],
    'city': ['NY', 'LA', 'CHI', 'SF', 'SEA']
})

tier_order = ['Low', 'Middle', 'High']
df['tier'] = pd.Categorical(df['tier'], categories=tier_order, ordered=True)

df_sorted = df.sort_values('tier')
```

↑ Back to top

Part 2. Pandas Basics

```
import pandas as pd
import numpy as np
```

The following describes the context of DataFrame in Part 2.

Open Payments, which is managed by the Centers for Medicare & Medicaid Services (CMS), is a national disclosure program created by the Affordable Care Act (ACA). The program promotes transparency and accountability by helping consumers understand the financial relationships between pharmaceutical and medical device industries, physicians, and non-physician practitioners (NPP). Non-physician practitioners (NPPs) collectively refers to physician assistants, nurse practitioners, clinical nurse specialists, certified registered nurse anesthetists/anesthesiologists and certified nurse-midwives provider types.

These financial relationships may include consulting fees, research grants, travel reimbursements, and payments made from the industry to medical practitioners. It is important to note that financial ties between the health care industry and health care providers do not necessarily indicate an improper relationship.

- Let us consider the CMS' Open Payments data for selected cities in **New York state** in **year 2022**, the `cms_ny_2022` DataFrame:

```
cms_ny_2022 = pd.read_csv('https://bcdanl.github.io/data/cms-2022-cities-all.zip'
```

Variable Description

variable	type	description
`Record_ID`	numeric	Unique identifier for a transaction between a manufacturer and a physician or an NPP
`Month`	nu	Month
`Day`	numeric	Year

↑ Back to top

variable	type	description
`Amount_of_Payment`	numeric	US dollar amount of payment to a physician or NPP
`Product`	chracter	An indicator showing if the product is a `Drug`, `Device`, `Biological`, or `Medical Supply`.
`Product_Manufacturer`	chracter	Manufacturer making the payment to a physician/NPP
`NPI`	numeric	National Provider Identifier, a unique identification number for a physician/NPP receiving the payment
`Physician_or_NPP`	chracter	An indicator showing if the recipient of the payment is a physician or non-physician practitioner (NPP)
`First_Name`	chracter	First name of physician/NPP in New York state
`Last_Name`	chracter	Last name of physician/NPP in New York state
`City`	chracter	City of the physician/NPP in New York state
`Type`	chracter	Type of physician/NPP (e.g., `Medical Doctor`, `Doctor of Optometry`, `Nurse Practitioner`, `Physician Assistant`, and more)
`Taxonomy`	chracter	Standardized taxonomy of medical service provider (e.g., `Allopathic & Osteopathic Physicians`, `Chiropractic Providers`, `Dental Providers`, `Physician Assistants & Advanced Practice Nursing Providers`, and more)

↑ Back to top

variable	type	description
`Specialty`	chracter	A specialty chosen within the standardized taxonomy (e.g., `Psychiatry & Neurology`, `Physician Assistant`, `Chiropractor`, `Dentist`, `Family Medicine`, `Neurological Surgery`, and more)
`Specialty_Detail`	chracter	Detail in the specialty if any (e.g., `Family`, `Gastroenterology`, `Cardiovascular Disease`, `Neurology`, `Psychiatry`, and more)

- Note that each physician/NPP corresponds to one single `Specialty_Detail` value.

Record_ID <dbl>	Month <dbl>	Day <dbl>	Amount_of_Payment <dbl>	Product <chr>
1	1	1	223.28	Device
2	1	1	23.32	Drug
3	1	1	164.86	Device
4	1	1	223.28	Device
5	1	1	19.40	Drug
6	1	1	43.87	Device
7	1	1	4.00	Device
8	1	1	108.44	Device
9	1	1	15.52	Device
10	1	1	223.28	Device

1-10 of 10,000 rows | 1-5 of 15 columns Previous 1 2 3 4 5 6 ... 1000 Next

Question 3

- Write a Python code to convert the st ian_or_NPP variable into the **category** type variable.

↑ Back to top

Question 4

- Write a Python code to replace the **NaN** value of the `Specialty_Detail` variable with the value of the `Specialty` variable.
  - For example, the following shows that
    - All **NaN** values in the `Specialty_Detail` variable, where the `Specialty` is **Physician Assistant**, are replaced with **Physician Assistant**.

Specialty <chr>	Specialty_Detail <chr>
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant
Physician Assistant	Physician Assistant

1-10 of 10,000 rows Previous 1 2 3 4 5 6 … 1000 Next

Question 5

Write a Python code to count the number of transactions in which the payment amount (i.e., `Amount_of_Payment`) falls in the top 10% of all payments in the DataFrame.

Question 6

Write a Python code to identify the city or cities with the highest number of non-missing values in the `Specialty_Detail` variable of the `cms_ny_2022` DataFrame.

↑ Back to top

Question 7

- Write a Python code to count the number of occurrences of each unique value in `Specialty_Detail` for each unique value in `Taxonomy`.
  - For example, the following displays the number of occurrences of each unique value in `Specialty_Detail` for the **Dental Providers** `Taxonomy` value.

Specialty_Detail	n
<chr>	<int>
Dentist	1394
Dentist Anesthesiologist	1
Endodontics	128
General Practice	1177
Oral and Maxillofacial Pathology	10
Oral and Maxillofacial Radiology	1
Oral and Maxillofacial Surgery	140
Orofacial Pain	1
Orthodontics and Dentofacial Orthopedics	472
Pediatric Dentistry	45

1-10 of 12 rows Previous 1 2 Next

Question 8

- Write a Python code to count the number of unique `Specialty_Detail` values for the value **"Dental Providers"** in `Taxonomy`.

Taxonomy	n_unique
<chr>	<int>
Dental Providers	12

1 row

Question 9

↑ Back to top

- Write a Python code (1) to count the number of missing values and (2) to calculate mean, standard deviation, minimum, first quartile, median, third quartile, and maximum of the

`Amount_of_Payment` variable for the value **Dermatology** in the `Specialty` variable.

Question 10

- Write a Python code to replicate the following DataFrame, which calculates (1) the number of physicians in each city, (2) the number of NPPs in each city, (3) the number of physicians and NPPs in each city, and (4) the percentage of physicians and NPPs in each city, as shown below.
  - Ensure that the sum of the percentages of physicians and NPPs within each city is 100.
  - Ensure that the observations are sorted by `Total_Count` and `City` in descending and ascending orders, respectively.

City	Physician_or_NPP	count	Total_Count	Percent
<chr>	<chr>	<int>	<int>	<dbl>
New York	NPP	1933	8819	21.92
New York	Physician	6886	8819	78.08
Brooklyn	NPP	1020	4594	22.20
Brooklyn	Physician	3574	4594	77.80
Bronx	NPP	555	2392	23.20
Bronx	Physician	1837	2392	76.80
Buffalo	NPP	665	1678	39.63
Buffalo	Physician	1013	1678	60.37
Rochester	NPP	524	1527	34.32
Rochester	Physician	1003	1527	65.68
Albany	NPP	357	1031	34.63
Albany	Physician	674	1031	65.37
Syracuse	NPP	363	912	39.80
Syracuse	Physician	549	912	60.20
Staten Island	NPP	222	887	25.03
Staten Island	Physician	665	887	74.97
Flushing	NPP	187	870	21.49
Flushing	Physician	683	870	78.51
Williamsville	NPP	311	791	39.32
Williamsville	Physician	480	791	60.68
New Hyde Park	NPP	196	787	24.90
New Hyde Park	Physician	591	787	75.10
Amherst	NPP	163	365	44.66
Amherst	Physician	202	365	55.34

↑ Back to top

1-24 of 24 rows

Question 11

- Write a Python code to calculate:
  - The total `Amount_of_Payment` the `Product_Manufacturer` **AbbVie Inc.** paid to physicians and non-physician practitioners (NPPs) in `Cities` of **Rochester**, **Buffalo**, and **Syracuse**.
  - How many times the `Product_Manufacturer` **AbbVie Inc.** paid to physicians and non-physician practitioners (NPPs) in in `Cities` of **Rochester**, **Buffalo**, and **Syracuse**.

Product_Manufacturer	Total_Amount_of_Payment	count
<chr>	<dbl>	<int>
AbbVie Inc.	117565.3	5633

1 row

Question 12

- Consider the following two DataFrames, `cms_ny_2022_npi` and `cms_ny_2022_records`:

```
cms_ny_2022_npi = pd.read_csv('https://bcdanl.github.io/data/cms-2022-cities-npi.
```

NPI	Physician_or_NPP	First_Name
<dbl>	<chr>	<chr>
1124540216	Physician	JONATHAN
1215112719	Physician	IRINA
1285855486	Physician	Amitabh
1376834739	Physician	BENJAMIN
1528188190	Physician	JOHANNA
1568653947	Physician	ANIL
1568882249	NPP	CHERIE
1609981513	Physician	Alexis
1639130925	Physician	TAK
1679705230	Physician	JOSHUA

↑ Back to top

1-10 of 10,000 rows | 1-3 of 9 columns Previous 1 2 3 4 5 6 … 1000 Next

- `cms_ny_2022_npi` : This DataFrame is created from `cms_ny_2022` by selecting only the columns `NPI`, `Physician_or_NPP`, `First_Name`, `Last_Name`, `City`, `Type`, `Taxonomy`, `Specialty`, and `Specialty_Detail`.
- In addition, it contains only unique observations (i.e., there are no duplicate rows).

```
cms_ny_2022_records = pd.read_csv('https://bcdanl.github.io/data/cms-2022-cities-
```

Record_ID <dbl>	Month <dbl>	Day <dbl>	Amount_of_Payment <dbl>	Product <chr>
1	1	1	223.28	Device
2	1	1	23.32	Drug
3	1	1	164.86	Device
4	1	1	223.28	Device
5	1	1	19.40	Drug
6	1	1	43.87	Device
7	1	1	4.00	Device
8	1	1	108.44	Device
9	1	1	15.52	Device
10	1	1	223.28	Device

1-10 of 10,000 rows | 1-5 of 7 columns Previous 1 2 3 4 5 6 … 1000 Next

- `cms_ny_2022_records` : This DataFrame is derived from `cms_ny_2022` by including every observation but only retaining the columns `Record_ID`, `Month`, `Day`, `Amount_of_Payment`, `Product`, `Product_Manufacturer`, and `NPI`.
- Write a Python code to create the `cms_ny_2022` DataFrame by using the `cms_ny_2022_npi` and `cms_ny_2022_records` DataFrames.

Question 13

- Write a Python code to create the following DataFrame, `City_Physician_or_NPP`, which counts the number of physicians and the non-physician practitioners (NPPs) in each city.

↑ Back to top

City	NPP	Physician
<chr>	<int>	<int>
Albany	357	674
Amherst	163	202
Bronx	555	1837
Brooklyn	1020	3574
Buffalo	665	1013
Flushing	187	683
New Hyde Park	196	591
New York	1933	6886
Rochester	524	1003
Staten Island	222	665
Syracuse	363	549
Williamsville	311	480

1-12 of 12 rows

powered with github, quarto, and rstudio
byeong-hak choe, 2025

↑ Back to top